Recover a pipeline when one step drifts

In this guide, you'll run a short pipeline, catch a bad middle step, and retry from the last good boundary. The retry cites the retained plan artifact and causally follows the failed draft and inspector — it does not write into an existing output as if it were a branch. The sample task is an infographic tile. The same pattern works for code generation, data transforms, migrations, and other pipelines where one bad step can spoil useful work.

Setup

Load the launch helpers.

In [1]:
```
import pathlib
import sys


def _find_visual_artifact_example_root():
    cwd = pathlib.Path.cwd().resolve()
    candidates = []
    for base in (cwd, *cwd.parents):
        candidates.append(base)
        candidates.append(base / "examples" / "notebooks" / "visual_artifact
    for candidate in candidates:
        if (candidate / "shepherd_usecases" / "visual_artifact" / "launch.py
            return candidate
    raise RuntimeError(
        "Cannot find examples/notebooks/visual_artifact. "
        "Launch JupyterLab from the repository root with `make notebooks`."
    )


example_root = _find_visual_artifact_example_root()
if str(example_root) not in sys.path:
    sys.path.insert(0, str(example_root))
try:
    from shepherd_usecases.visual_artifact import launch
    from shepherd_usecases.visual_artifact import viz
except Exception as exc:
    raise RuntimeError(
        "Could not import the visual-artifact notebook helpers. "
        "Launch JupyterLab from the repository root with `make notebooks`."
    ) from exc

launch.bootstrap(example_root=example_root)
```

What this run does

Pipeline Recovery runs a two-step pipeline: a plan, then a draft. A reviewer catches the draft going wrong, an inspector reads the trace to diagnose it, and the draft is retried as a new run that cites the retained plan artifact (`plan_ref`), the last good boundary.

The input

For this example, the run starts from one plain prompt. `launch.plan_for` turns it into a `brief` and an initial `plan` ; both are used as inputs to the pipeline's first task.

In [2]:
```
prompt = launch.default_prompt()
brief, plan = launch.plan_for(prompt)
plan
```

Out[2]:
```
{'artifact': 'gradient_descent_tile',
 'framing': 'contour-map',
 'request': 'Please output an infographic tile explaining gradient descen
t.',
 'must_include': ['Gradient Descent',
  'Start',
  'Step',
  'Minimum',
  'Small downhill steps reduce error.'],
 'decision_critical': 'update path must descend toward the minimum'}
```

Run Shepherd

First, open a workspace for the prompt. Every run in this guide belongs to it.

In [3]:
```
workspace = launch.open_workspace("visual-pipeline-recovery", prompt=prompt,
```

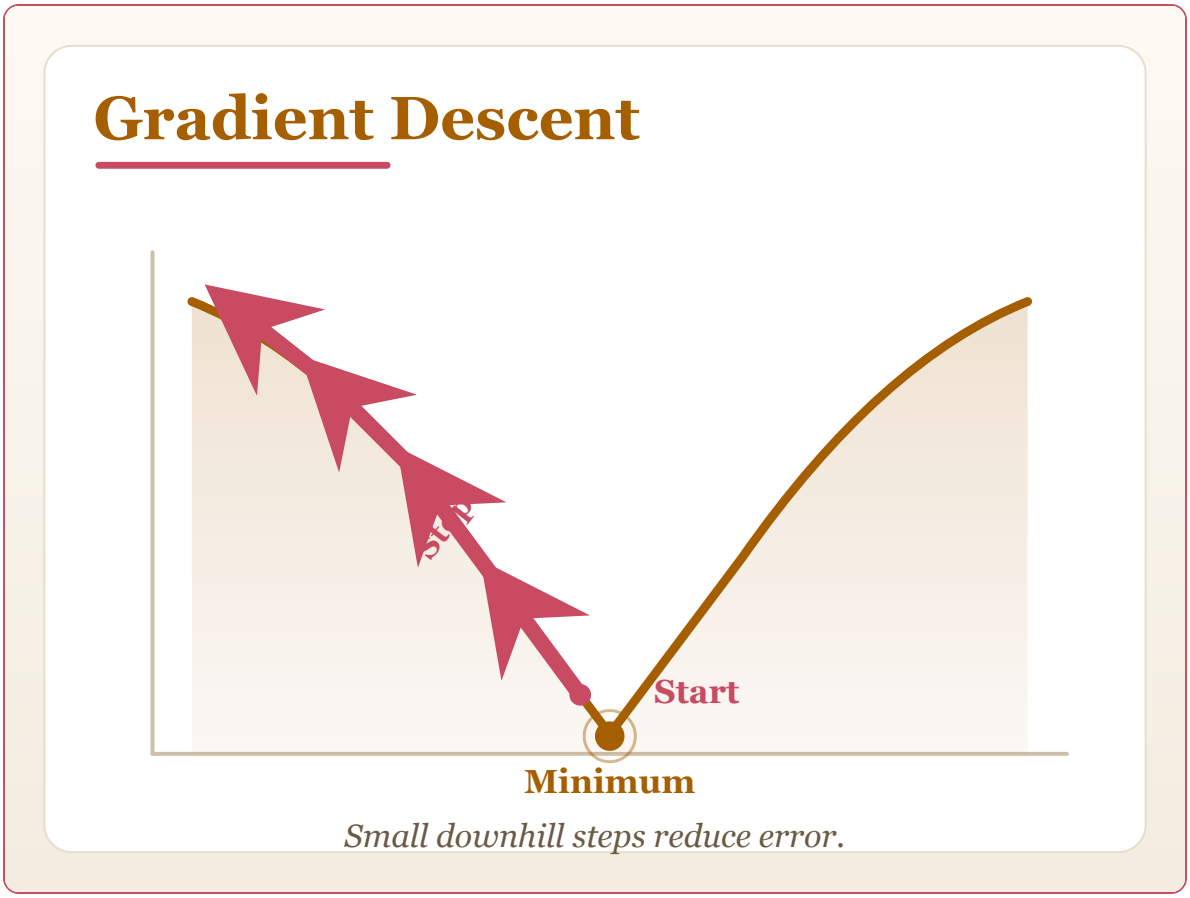
Now run the pipeline: first the plan, then the draft. The plan is the last good boundary; the draft is the step that drifts.

In [4]:
```
plan_run = launch.run_static(
    workspace,
    name="plan",
    output_path=launch.PLAN_PATH,
    output_content=plan,
    metadata={"logical_boundary": "plan"},
)
plan_ref = launch.artifact_ref(plan_run, launch.PLAN_PATH, label="retry-plar

draft_v1 = launch.run_with_artifact_ref(
    workspace,
    name="draft-v1",
    ref_name="plan",
    artifact_ref=plan_ref,
    output_path=launch.ARTIFACT_PATH,
    output_text=launch.draft_html(brief, corrupt=True),
    after=[plan_run],
```

```
    metadata={"failed_run": "draft-v1"},
)
viz.show(viz.run_artifact(draft_v1, label="draft v1: wrong direction", accer
```

draft v1: wrong direction



Two tasks examine the failed draft. First a reviewer flags it — an agent that reads the draft and judges whether it met the brief. Then an inspector reads the reviewer's verdict and writes a structured diagnosis: what drifted and what to change on retry. The reviewer's output is retained read-only through `review_ref`; the inspector does not write into it.

In [5]:
```
draft_ref = launch.artifact_ref(draft_v1, label="failed-draft")
reviewer = launch.run_with_artifact_ref(
    workspace,
    name="review",
    ref_name="candidate",
    artifact_ref=draft_ref,
    output_path=launch.VERDICT_PATH,
    output_content=launch.review_content(prompt, {"draft_v1": draft_v1}),
    after=[draft_v1],
)
selection = launch.selection_from_review(reviewer)
issues = list(selection.candidates[0].get("issues", []))
review_ref = launch.artifact_ref(reviewer, launch.VERDICT_PATH, label="draft
inspector = launch.run_with_artifact_ref(
    workspace,
    name="inspector",
```

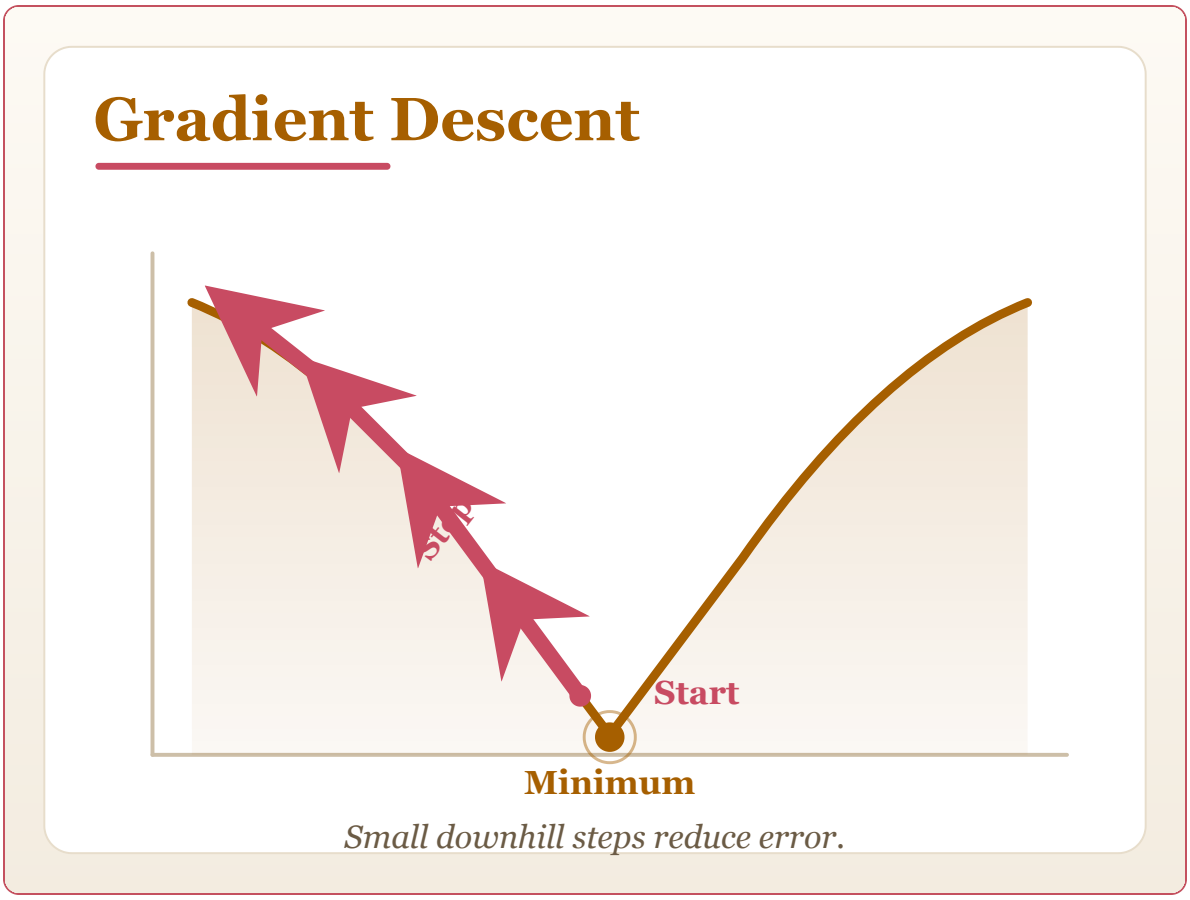
```
    ref_name="review",
    artifact_ref=review_ref,
    output_path=launch.DIAGNOSIS_PATH,
    output_content=launch.diagnosis_content(issues),
    after=[reviewer],
)
launch.read_json(inspector, launch.DIAGNOSIS_PATH)
```

Out[5]:
```
{'bad_step': 'draft',
 'evidence': 'index.html -> descent path moves uphill away from the minimu
m',
 'failure_type': 'wrong_direction',
 'recommended_change': 'Reverse the update path so each step descends towar
d the minimum.',
 'retry_boundary': 'after plan, before draft'}
```

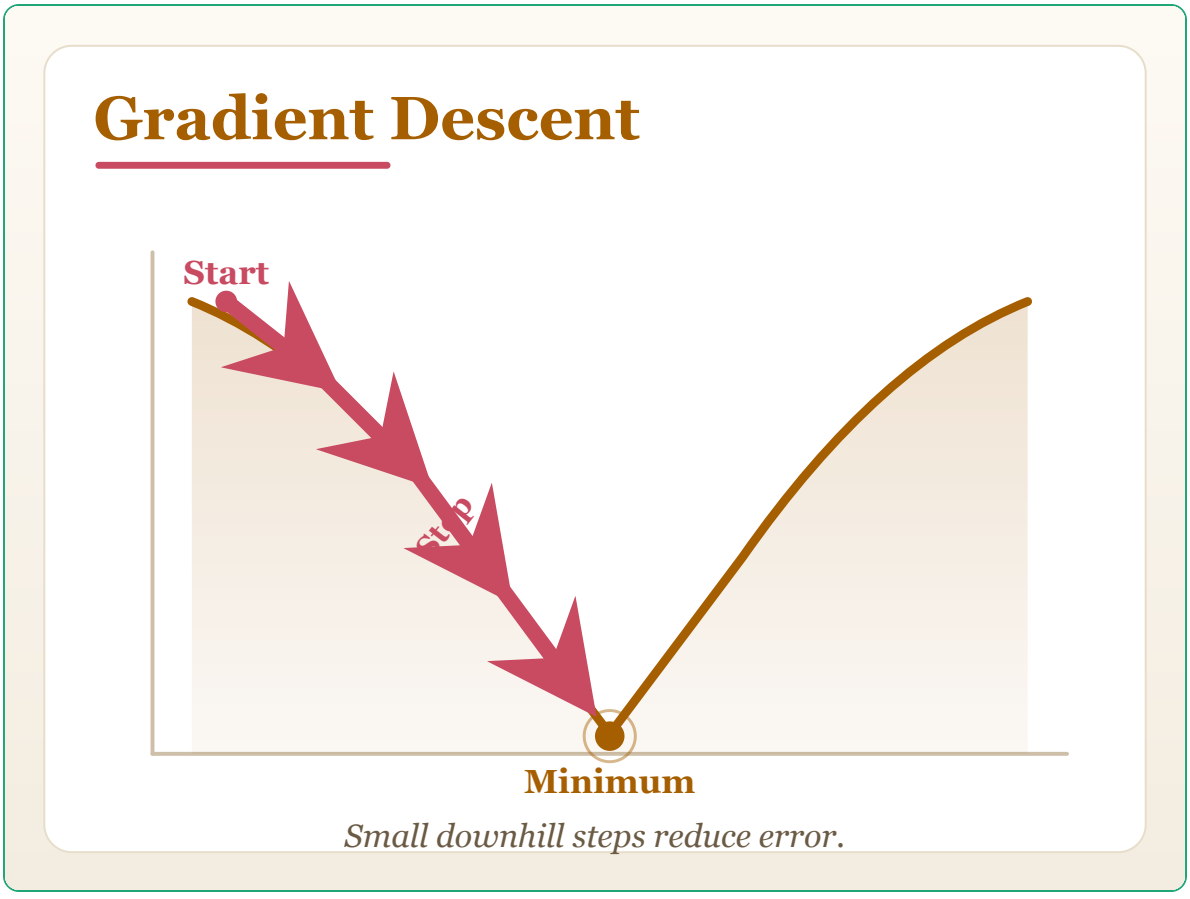
The inspector's diagnosis is packaged as `diagnosis_ref`. The retry is a new run — not a branch of an existing one. It cites both `plan_ref` (the retained plan artifact, the last good boundary) and `diagnosis_ref` as artifact refs, and declares `after=[plan_run, inspector]` so the workspace trace records the causal chain. The failed draft stays untouched as evidence.

In [6]:
```
diagnosis_ref = launch.artifact_ref(inspector, launch.DIAGNOSIS_PATH, label=
retry = launch.run_with_artifact_refs(
    workspace,
    name="retry",
    refs={"plan": plan_ref, "diagnosis": diagnosis_ref},
    output_path=launch.ARTIFACT_PATH,
    output_text=launch.draft_html(brief, corrupt=False),
    after=[plan_run, inspector],
    metadata={"retry_run": "retry-from-plan"},
)
viz.show(viz.side_by_side([
    viz.run_artifact(draft_v1, label="failed draft", accent="red"),
    viz.run_artifact(retry, label="retry from cited plan", accent="green"),
]))
```

failed draft



retry from cited plan



The two draft outputs now sit side by side: the failed run is still available as evidence, and the retry is the result of the new run with the fix applied. Now formalize the decision: discard the failed draft, release the supporting runs, and select the retry.

In [7]:
```
draft_v1.output().discard()
plan_run.output().release()
reviewer.output().release()
inspector.output().release()
retry.output().select()
```

Out[7]:
```
RetainedOutputSelectionResult(scope=ScopeInfo(name='run-9937b9467b22', ref
='refs/metagit/scopes/run-9937b9467b22', instance_id='248789ad9636', creati
on_oid='38feb2bbbf529566ffa64ce313c3fd4360698832', world_id='world_f0940c98
2c36'), parent=ScopeInfo(name='ground', ref='refs/metagit/ground', instance
_id='ground-ebe9047c54bf', creation_oid='', world_id='world_8b4355f1f36b'),
output_world_oid='845b4817def289817597516a91f5c701b84807d9', parent_world_b
efore='0e818ddeb541e1a8287537c64be1e4bffe3b7b78', parent_world_after='fee8c
4a72c4e79e763a5625e974e26323e920fad', settlement=RetainedOutputSettlement(s
cope_name='run-9937b9467b22', scope_ref='refs/metagit/scopes/run-9937b9467b
22', scope_instance_id='248789ad9636', parent_ref='refs/metagit/ground', ha
ndoff_ref='refs/metagit/seals/b64u_cnVuLTk5MzdiOTQ2N2IyMg/b64u_MjQ4Nzg5YWQ5
NjM2', output_world_oid='845b4817def289817597516a91f5c701b84807d9', binding
='workspace', store_id='store_workspace', resource_id='fs:repo-main', candi
date_id='primary', candidate_head='61ba6f5df48bb82578df37c762b1883f35185b6
3', action='selected', operation_id='select_retained_5a27e4668a3bbc1102ac03
33eccb37db', parent_world_before='0e818ddeb541e1a8287537c64be1e4bffe3b7b7
8', parent_world_after='fee8c4a72c4e79e763a5625e974e26323e920fad', settleme
nt_ref='refs/metagit/settlements/b64u_cnVuLTk5MzdiOTQ2N2IyMg/b64u_MjQ4Nzg5Y
WQ5NjM2/b64u_d29ya3NwYWNl/b64u_cHJpbWFyeQ', p030_authority_operation_id='op
_8d42e2c1d200', p030_authority_settlement_operation_id='op_8d42e2c1d200_set
tlement', p030_authority_outcome='allowed'), p030_authority_operation_id='o
p_8d42e2c1d200', p030_authority_settlement_operation_id='op_8d42e2c1d200_se
ttlement', p030_authority_outcome='allowed')
```

Trace

`workspace.flow` captures every action taken across all runs above, plus the causal relationships among them — including the link from `plan_run` and `inspector` to `retry`. Click the nodes to explore individual events.

In [8]:
```
viz.show(viz.trace(
    workspace.flow,
    {"plan": plan_run, "draft_v1": draft_v1, "review": reviewer, "inspector"
    height="680px",
    detail="events",
))
```

Events

kind	run/flow	label	payload
flow.opened	flow-b94c7b3c7998		name='visual-pipeline-recovery
flow.fork.requested	run-bf8a267ff61f	plan	name='plan', after=[]
flow.logical_boundary	run-bf8a267ff61f	plan	label='plan'
run.lifecycle	run-bf8a267ff61f	plan	task_id='shepherd_usecases.vis enforcement='advisory'
provider.invocation	run-bf8a267ff61f	plan	execution_id='task-execution-5 provider_event_kind='provider. evidence_role='provider_proven 'sha256:b5953c66b8b084e3ea1327 'shepherd_usecases.visual_arti
provider.invocation	run-bf8a267ff61f	plan	execution_id='task-execution-5 provider_event_kind='provider. evidence_role='provider_proven 'plan.json', 'content_digest': 'launched_confined': False}
run.output.published	run-bf8a267ff61f	plan	output_name='workspace', outpu binding='workspace', output_wo
run.output.settled	run-bf8a267ff61f	plan	output_name='workspace', outpu state='released', settlement_ref='refs/metagit/s
flow.fork.requested	run-c420d4b847f2	draft_v1	name='draft-v1', after=['run-b
flow.failed_run	run-c420d4b847f2	draft_v1	label='draft-v1'
run.lifecycle	run-c420d4b847f2	draft_v1	task_id='shepherd_usecases.vis enforcement='advisory'
provider.invocation	run-c420d4b847f2	draft_v1	execution_id='task-execution-7 provider_event_kind='provider. evidence_role='provider_proven 'sha256:191dac3b7761d75c191991 'shepherd_usecases.visual_arti

In [9]:
```
workspace.close()
```

Want to understand how it works?

If you want to understand how this works in greater detail, open the Pipeline Recovery internals notebook.

Next steps

- Find the best approach by trying several alternatives

- Find the cheapest model that still does the job